OS BASICS

Process Scheduling

The Process scheduling

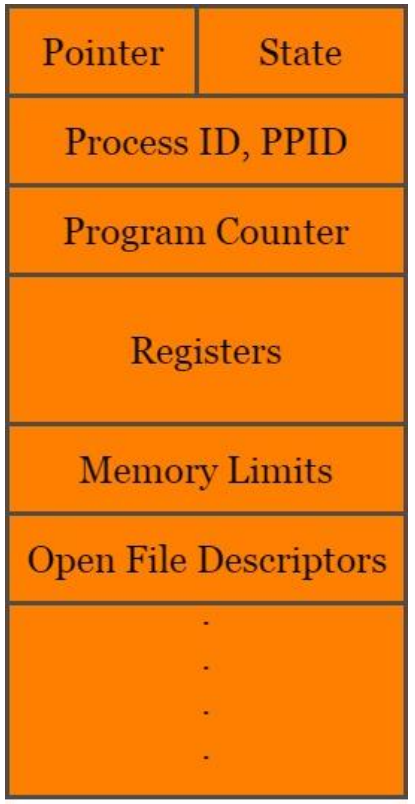
- In a single-processor system, only one process can run at a time; any others must wait(starve) until the CPU is free and can be rescheduled.
- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.
- The idea is relatively simple. A process is executed until it must wait, typically for the completion of some I/O request. In a simple computer system, the CPU then just sits idle. All this waiting time is wasted; no useful work is accomplished.
- With multiprogramming, you try to use this time productively. Several processes are kept in memory at one time.
- When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process. This pattern continues.
- Every time one process has to wait, another process can take over use of the CPU.
- The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy

- CPU scheduling decisions may take place when a process:
  - Switches from running to waiting state (for example, as the result of an I/O request or an invocation of wait for the termination of one of the child processes)
  - Switches from running to ready state (for example, when an interrupt occurs)
  - Switches from waiting to ready (for example, at completion of I/O)
  - Terminates
- For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution.
- There is a choice, however, for situations 2 and 3.
- When scheduling takes place only under circumstances 1 and 4, you say that the scheduling scheme is non-preemptive or cooperative; otherwise, it is preemptive.
- Under non-preemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.

Process Control Block (PCB):

Whenever a process is created, for every process PM (process manager) will create a look up table or say a data structure called process control block (PCB) in kernel space. There is a Process Control Block for each process, enclosing all the information about the process. It is a data structure, which contains the following information about a process:

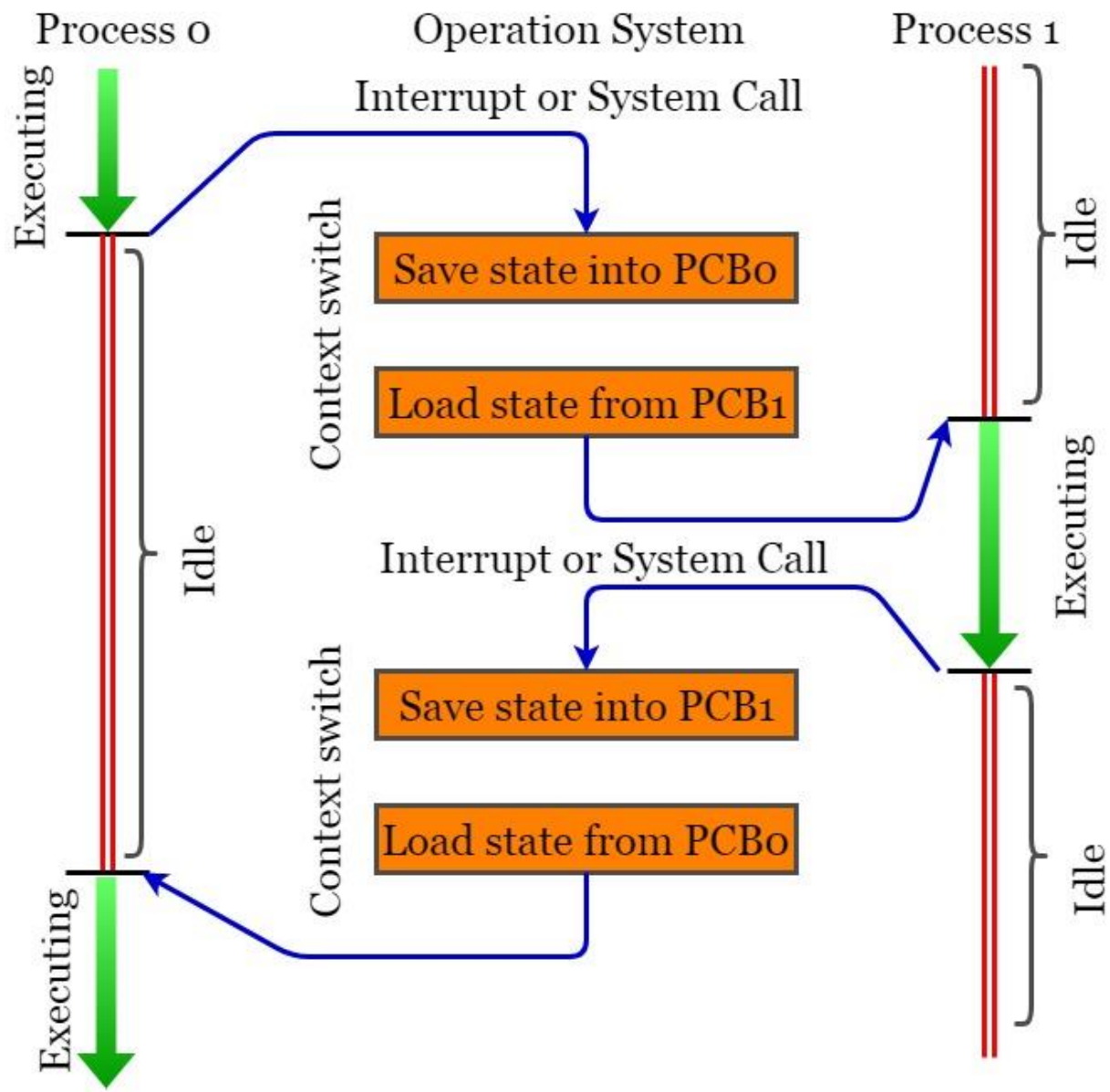
- Process state
- Proves ID, its Parent ID
- Program counter
- CPU registers
- CPU scheduling information
- Memory-management information
- List of open file descriptors
- Accounting information
- I/O status information



Process Control Block

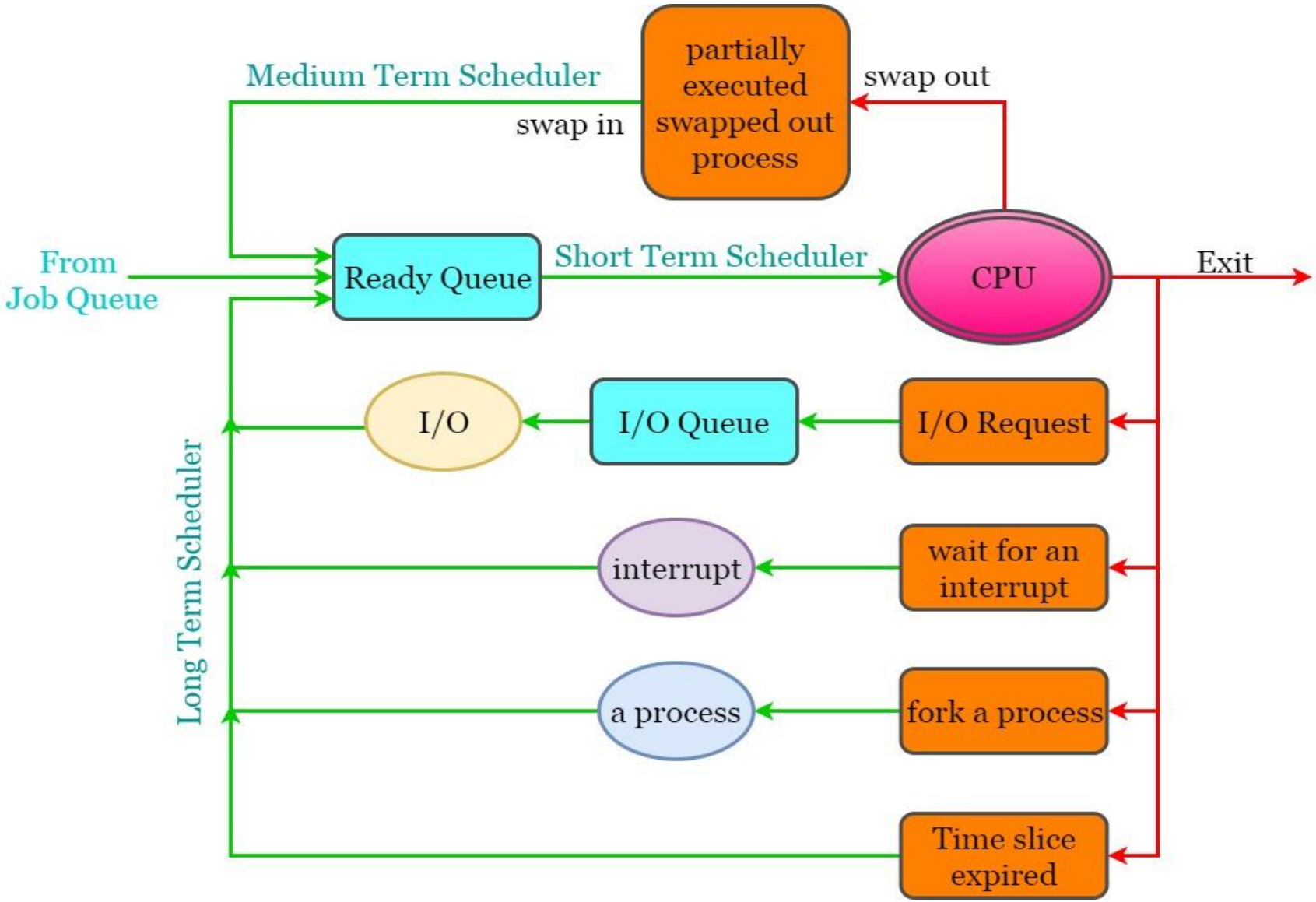
Context Switch:

- A context switch is the mechanism to store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time.
- Using this technique, a context switcher enables multiple processes to share a single CPU.
- Context switching is an essential part of a multitasking operating system features.
- When the scheduler switches the CPU from executing one process to execute another, the state from the current running process is stored into the process control block.
- After this, the state for the process to run next is loaded from its own PCB and used to set the PC, registers, etc. At that point, the second process can start executing.
- Context switch takes some time. For process context switch is overhead, because during context switch no process is executing. But context switch is done to give fare share of CPU to all processes.
- The context switch time depends on low level hardware also.


Process 0
Operation System
Process 1
Executing
Interrupt or System Call
Context switch
Save state into PCB0
Load state from PCB1
Idle
Idle
Executing
Interrupt or System Call
Context switch
Save state into PCB1
Load state from PCB0
Executing
Idle

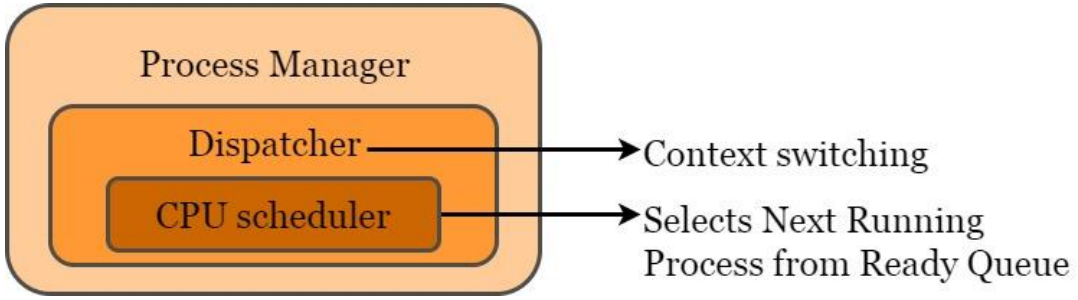
The Scheduling Queues:

- The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue.
- When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.
- The Operating System maintains the following important process scheduling queues
  - Job queue: This queue keeps all the processes once submitted in the system.
  - Ready queue: This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
  - Device queues: The processes which are blocked due to unavailability of an I/O device constitute this queue. Here are unique device queues for each I/O device available.


Medium Term Scheduler
partially executed swapped out process
swap out
swap in
From Job Queue
Ready Queue
Short Term Scheduler
CPU
Exit
I/O
I/O Queue
I/O Request
Long Term Scheduler
interrupt
wait for an interrupt
a process
fork a process
Time slice expired

The Dispatcher

- Another component involved in the CPU-scheduling function is the dispatcher. The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. This function involves the following:
- Switching context
- Switching to user mode
- Jumping to the proper location in the user program to restart that program
- The dispatcher should be as fast as possible, since it is invoked during every process switch. The time it takes for the dispatcher to stop one process and start another running is known as the dispatch latency.
- The kernel creates following three processes as a part of bootstrapping.
  - The process with PID 0 is the scheduler or swapper process.
  - The process with PID 1 is INIT
  - The process with PID 2 is pagedaemon



Attributes for CPU Scheduling:

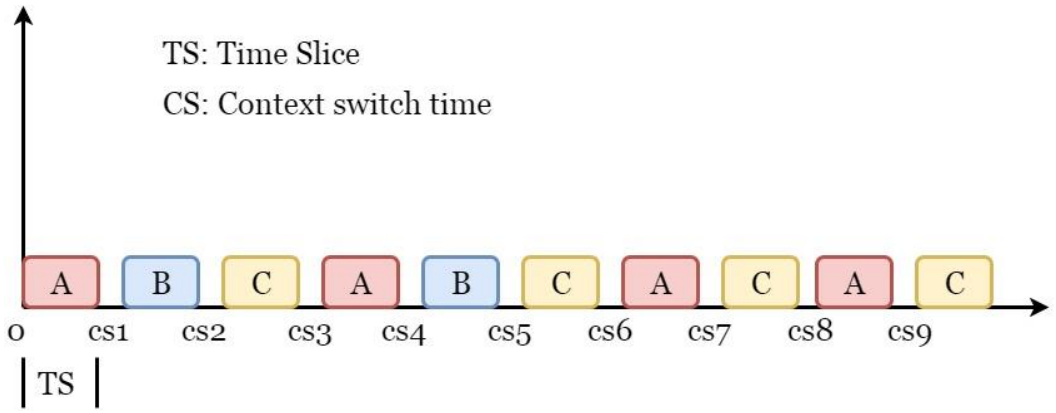
While selecting the scheduling policies you have to take care of following attributes of scheduling policies.

- Response Time: The time gap between the process loaded into the RAM and its first instruction executed by the CPU. Response time is the time from the submission of a request until the first response is produced.
- Starvation Time: A process in its life cycle how much starved without executing by CPU, this time is starvation time. The CPU scheduling algorithm does not affect the amount of time during which a process executes or does I/O; it affects only the amount of time that a process spends waiting in the ready queue. Starvation time or waiting time is the sum of the periods spent waiting in the ready queue. Starvation time includes response time.
- Turnaround Time: Turnaround time is a time gap between process loaded into the RAM and its execution completed. This is process life time. Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O. Turnaround time includes starvation time.
- Throughput: If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput.

- CPU Utilization: To make out the best use of CPU and not to waste any CPU cycle, CPU would be working most of the time (Ideally 100% of the time). Considering a real system, CPU usage should range from 40% (lightly loaded) to 90% (heavily loaded.)

	Process A	Process B	Process C
Response Time	0	1TS+1CS	2TS+2CS
Wait Time	5TS+8CS	3TS+4CS	6TS+9CS
Turnaround Time	9TS+8CS	5TS+4CS	10TS+9CS

- Fairness: In the example of processes A, B and C you can understand the concept fairness. Here if time slice is doubled, response time increases which is not desirable. And if time slice is halved, number of context switches increases so wait time increases and overhead increases. This is also not desirable. So fairness is nothing but desirable with parameters and for all processes.



Scheduling Algorithm Optimization Criteria

- Maximum CPU utilization
- Maximum throughput
- Minimum turnaround time
- Minimum waiting time
- Minimum response time
- More fairness is good
- **Scheduling Algorithms**
- A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms. Many CPU scheduling algorithms have both preemptive and non-preemptive versions.
- Preemptive: schedule a new process even when the current process does not intend to give up the CPU
- Non-preemptive: only schedule a new process when the current one does not want CPU any more.